```
import pandas as pd
from google.colab import files

# Upload the downloaded dataset
uploaded = files.upload()
```

Choose Files Telco-Custo...-Churn.csv

Telco-Customer-Churn.csv(text/csv) - 970457 bytes, last modified: n/a - 100% done

```
Saving Telco-Customer-Churn.csv to Telco-Customer-Churn.csv
```

```
# Read the uploaded CSV file into a DataFrame
file_name = next(iter(uploaded.keys()))

df = pd.read_csv(file_name)

# Display the dataset
print("Dataset converted into DataFrame successfully!")
print("Dataset shape:", df.shape)

df.head()
```

```
Dataset converted into DataFrame successfully!
Dataset shape: (7043, 21)
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	Pho
0	7590-VHVEG	Female	0	Yes	No	1	
1	5575-GNVDE	Male	0	No	No	34	
2	3668-QPYBK	Male	0	No	No	2	
3	7795-CFOCW	Male	0	No	No	45	
4	9237-HQITU	Female	0	No	No	2	

5 rows × 21 columns

```
# Display dataset information
print("Column Names:")
print(df.columns.tolist())

print("\nDataset Information:")
df.info()

print("\nFirst 5 Rows:")
display(df.head())

print("\nMissing Values:")
print(df.isnull().sum())
```

```
Column Names:
['customerID', 'gender', 'SeniorCitizen', 'Partner', 'Dependents', '

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   customerID        7043 non-null   object
 1   gender            7043 non-null   object
 2   SeniorCitizen     7043 non-null   int64
 3   Partner           7043 non-null   object
 4   Dependents        7043 non-null   object
 5   tenure            7043 non-null   int64
 6   PhoneService      7043 non-null   object
 7   MultipleLines     7043 non-null   object
 8   InternetService   7043 non-null   object
 9   OnlineSecurity    7043 non-null   object
 10  OnlineBackup      7043 non-null   object
 11  DeviceProtection  7043 non-null   object
 12  TechSupport       7043 non-null   object
 13  StreamingTV       7043 non-null   object
 14  StreamingMovies   7043 non-null   object
 15  Contract          7043 non-null   object
 16  PaperlessBilling  7043 non-null   object
 17  PaymentMethod     7043 non-null   object
 18  MonthlyCharges    7043 non-null   float64
 19  TotalCharges      7043 non-null   object
 20  Churn             7043 non-null   object
dtypes: float64(1), int64(2), object(18)
memory usage: 1.1+ MB

First 5 Rows:
```

customerID	gender	SeniorCitizen	Partner	Dependents	tenure	Pho

0	7590-VHVEG	Female		0	Yes	No	1
1	5575-GNVDE	Male		0	No	No	34
2	3668-QPYBK	Male		0	No	No	2
3	7795-CFOCW	Male		0	No	No	45
4	9237-HQITU	Female		0	No	No	2

5 rows × 21 columns

```
Missing Values:
customerID          0
gender              0
SeniorCitizen       0
Partner             0
Dependents          0
tenure              0
PhoneService        0
MultipleLines       0
InternetService     0
OnlineSecurity      0
OnlineBackup        0
DeviceProtection    0
TechSupport         0
StreamingTV         0
StreamingMovies     0
Contract            0
PaperlessBilling    0
PaymentMethod       0
MonthlyCharges      0
TotalCharges        0
Churn               0
dtype: int64
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from IPython.display import display
```

```
# CUSTOMER CHURN CLASSIFIER – COMPLETE MACHINE LEARNING PIPELINE
```

```
# CUSTOMER CHURN CLASSIFIER – COMPLETE MACHINE LEARNING PIPELINE

# ==========================================
# STEP 3: DATA CLEANING
# ==========================================

print("=" * 50)
print("DATA CLEANING")
print("=" * 50)

print("Duplicate rows:", df.duplicated().sum())
df = df.drop_duplicates().copy()

# Convert TotalCharges to numeric if available
if "TotalCharges" in df.columns:
    df["TotalCharges"] = pd.to_numeric(
        df["TotalCharges"], errors="coerce"
    )
    df["TotalCharges"] = df["TotalCharges"].fillna(0)

# Remove customer ID if available
if "customerID" in df.columns:
    df = df.drop(columns=["customerID"])

print("Dataset shape after cleaning:", df.shape)
print("Remaining missing values:", df.isnull().sum().sum())


# ==========================================
# STEP 4: TARGET VARIABLE ANALYSIS
# ==========================================

print("\n" + "=" * 50)
print("CUSTOMER CHURN DISTRIBUTION")
print("=" * 50)

if "Churn" not in df.columns:
    raise ValueError("Dataset must contain a 'Churn' column.")

print(df["Churn"].value_counts())
print("\nChurn percentages:")
print((df["Churn"].value_counts(normalize=True) * 100).round(2))

plt.figure(figsize=(6, 4))
sns.countplot(data=df, x="Churn")
plt.title("Customer Churn Distribution")
plt.xlabel("Churn")
plt.ylabel("Number of Customers")
plt.show()
```

```
# ========================================
# STEP 5: EXPLORATORY DATA ANALYSIS
# ========================================

print("\n" + "=" * 50)
print("EXPLORATORY DATA ANALYSIS")
print("=" * 50)

numeric_columns = df.select_dtypes(include=np.number).columns
display(df[numeric_columns].describe())

if "tenure" in df.columns:
    plt.figure(figsize=(8, 5))
    sns.histplot(
        data=df,
        x="tenure",
        hue="Churn",
        bins=20,
        multiple="stack"
    )
    plt.title("Customer Tenure vs Churn")
    plt.show()

if "MonthlyCharges" in df.columns:
    plt.figure(figsize=(8, 5))
    sns.boxplot(data=df, x="Churn", y="MonthlyCharges")
    plt.title("Monthly Charges vs Churn")
    plt.show()


# ========================================
# STEP 6: PREPARE FEATURES AND TARGET
# ========================================

print("\n" + "=" * 50)
print("FEATURE PREPARATION")
print("=" * 50)

X = df.drop(columns=["Churn"])
y = df["Churn"].map({"Yes": 1, "No": 0})

if y.isnull().any():
    raise ValueError(
        "Unexpected labels in Churn. Check your dataset's target val
    )
```

```
X = pd.get_dummies(X, drop_first=True, dtype=int)

print("Features shape:", X.shape)
print("Target shape:", y.shape)


# ========================================
# STEP 7: TRAIN-TEST SPLIT
# ========================================

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("Training records:", X_train.shape[0])
print("Testing records:", X_test.shape[0])


# ========================================
# STEP 8: TRAIN MACHINE LEARNING MODELS
# ========================================

from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier

log_model = make_pipeline(
    StandardScaler(),
    LogisticRegression(max_iter=2000, random_state=42)
)

rf_model = RandomForestClassifier(
    n_estimators=200,
    random_state=42,
    class_weight="balanced"
)

log_model.fit(X_train, y_train)
rf_model.fit(X_train, y_train)

print("\nLogistic Regression trained successfully!")
```

```
print("Random Forest trained successfully!")


# ========================================
# STEP 9: MODEL EVALUATION
# ========================================

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    classification_report
)

models = {
    "Logistic Regression": log_model,
    "Random Forest": rf_model
}

results = []

for name, model in models.items():
    predictions = model.predict(X_test)
    probabilities = model.predict_proba(X_test)[:, 1]

    results.append({
        "Model": name,
        "Accuracy": accuracy_score(y_test, predictions),
        "Precision": precision_score(
            y_test, predictions, zero_division=0
        ),
        "Recall": recall_score(
            y_test, predictions, zero_division=0
        ),
        "F1 Score": f1_score(
            y_test, predictions, zero_division=0
        ),
        "ROC-AUC": roc_auc_score(y_test, probabilities)
    })

    print(f"\nClassification Report: {name}")
    print(classification_report(
        y_test,
        predictions,
        target_names=["No Churn", "Churn"],
        zero_division=0
    ))
```

```
    ''

results_df = pd.DataFrame(results)

print("\nMODEL COMPARISON")
display(results_df.round(3))


# ========================================
# STEP 10: CONFUSION MATRIX
# ========================================

from sklearn.metrics import confusion_matrix

rf_predictions = rf_model.predict(X_test)
cm = confusion_matrix(y_test, rf_predictions)

plt.figure(figsize=(6, 4))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=["No Churn", "Churn"],
    yticklabels=["No Churn", "Churn"]
)

plt.title("Random Forest Confusion Matrix")
plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")
plt.show()


# ========================================
# STEP 11: FEATURE IMPORTANCE
# ========================================

importance = pd.Series(
    rf_model.feature_importances_,
    index=X_train.columns
).sort_values(ascending=False)

print("TOP 10 IMPORTANT FEATURES")
display(importance.head(10).to_frame("Importance"))

importance.head(10).sort_values().plot(
    kind="barh",
    figsize=(9, 5)
)
```

```
plt.title("Top 10 Feature Importances")
plt.xlabel("Importance")
plt.show()


# ========================================
# STEP 12: FINAL PROJECT SUMMARY
# ========================================

best_model_row = results_df.loc[
    results_df["F1 Score"].idxmax()
]

print("\n" + "=" * 50)
print("CUSTOMER CHURN CLASSIFIER - PROJECT SUMMARY")
print("=" * 50)

print("Total customers:", len(df))
print("Training records:", len(X_train))
print("Testing records:", len(X_test))
print("Best model by F1-score:", best_model_row["Model"])
print("Accuracy:", round(best_model_row["Accuracy"], 3))
print("Precision:", round(best_model_row["Precision"], 3))
print("Recall:", round(best_model_row["Recall"], 3))
print("F1-score:", round(best_model_row["F1 Score"], 3))
print("ROC-AUC:", round(best_model_row["ROC-AUC"], 3))


# ========================================
# STEP 13: SAVE MODEL COMPARISON RESULTS
# ========================================

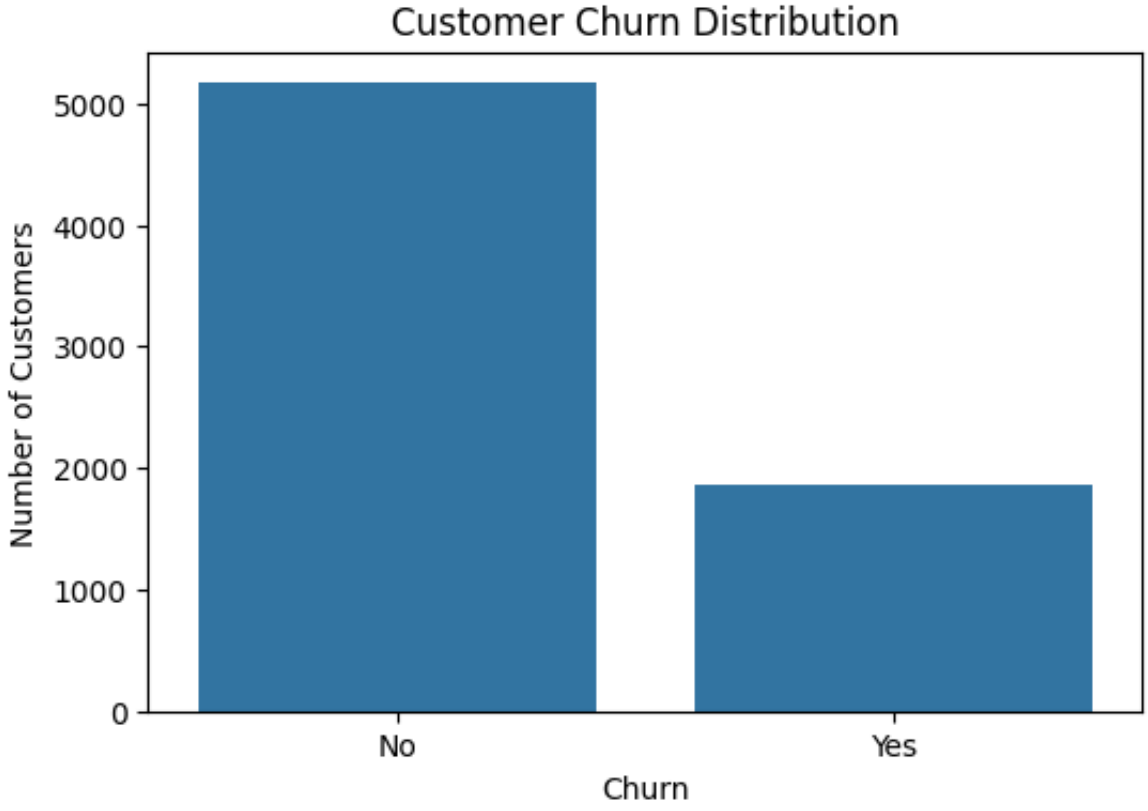
results_df.to_csv(
    "churn_model_comparison.csv",
    index=False
)

print("\nModel comparison saved as churn_model_comparison.csv")
print("Complete machine learning pipeline executed successfully!")
```

```
==================================================
DATA CLEANING
==================================================
Duplicate rows: 0
Dataset shape after cleaning: (7021, 20)
Remaining missing values: 0
```

```
==================================================
CUSTOMER CHURN DISTRIBUTION
==================================================
Churn
No     5164
Yes    1857
Name: count, dtype: int64

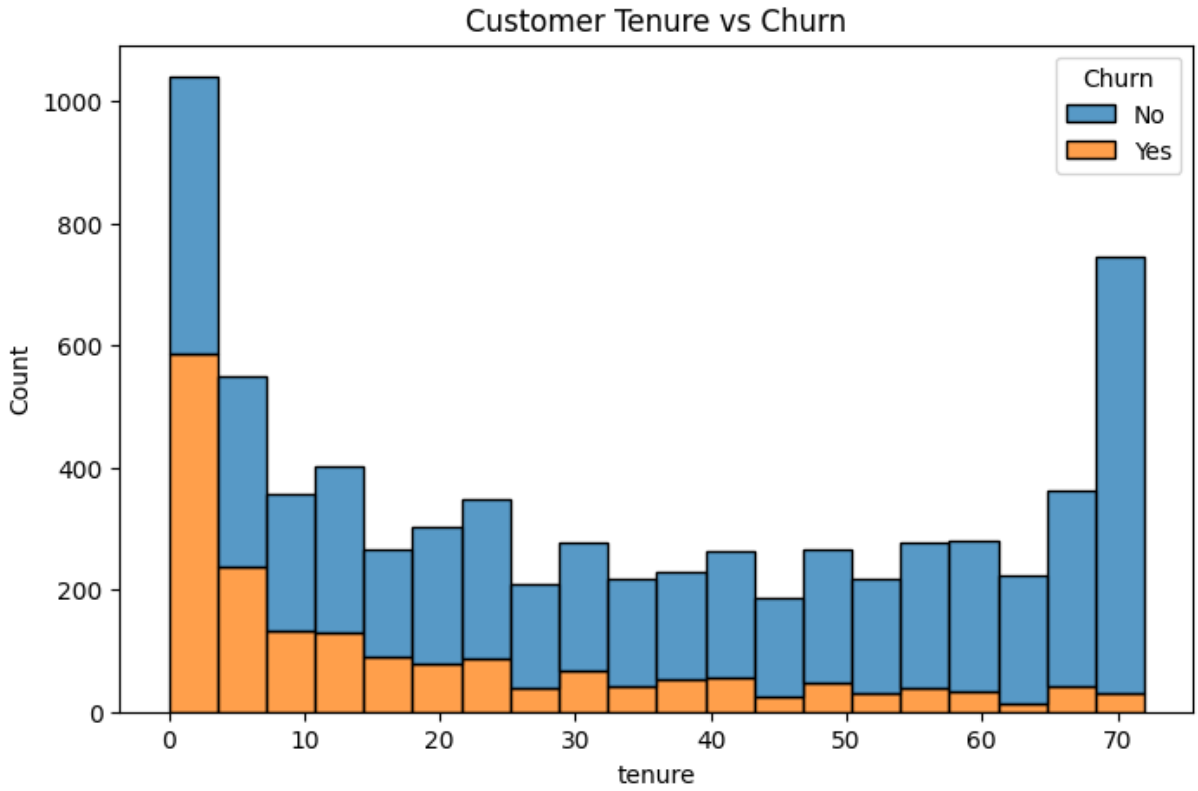
Churn percentages:
Churn
No     73.55
Yes    26.45
Name: proportion, dtype: float64
```

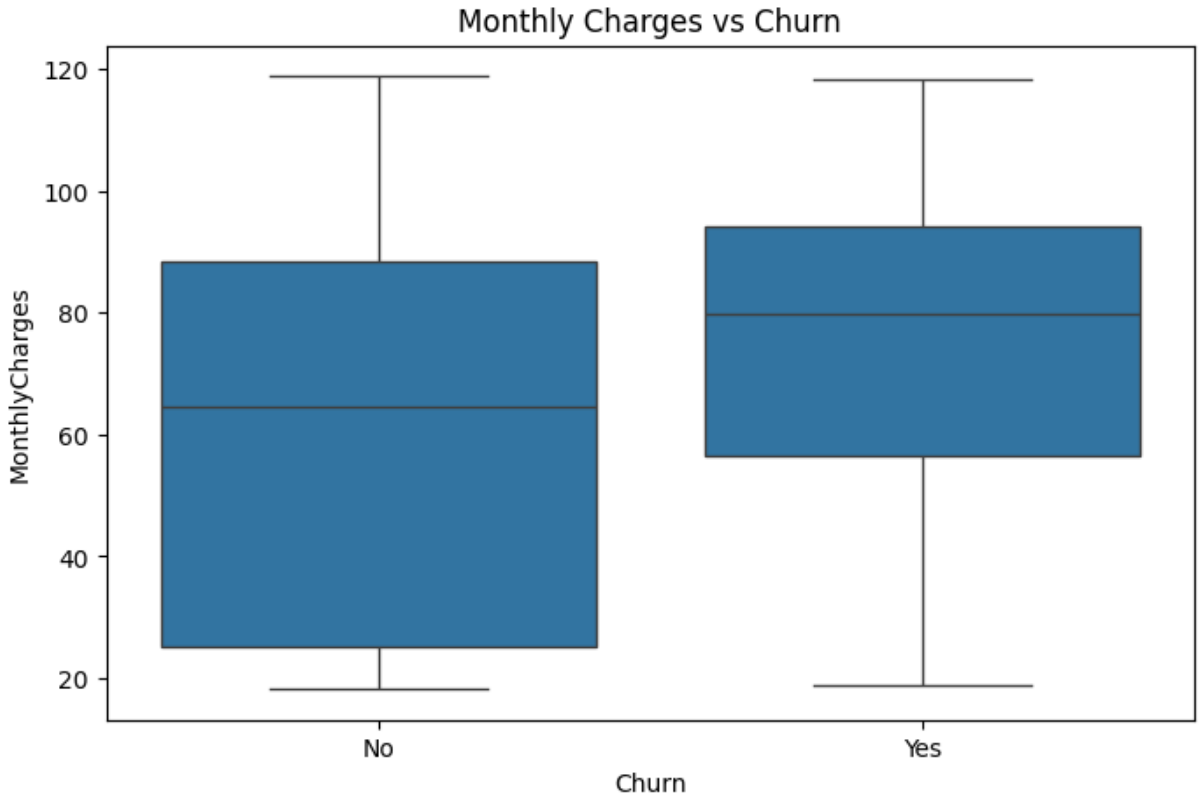



```
==================================================
EXPLORATORY DATA ANALYSIS
==================================================
```

	SeniorCitizen	tenure	MonthlyCharges	TotalCharges
count	7021.000000	7021.000000	7021.000000	7021.000000
mean	0.162512	32.469449	64.851894	2286.765026
std	0.368947	24.534965	30.069001	2266.855057
min	0.000000	0.000000	18.250000	0.000000
25%	0.000000	9.000000	35.750000	403.350000
50%	0.000000	29.000000	70.400000	1400.550000

75%	0.000000	55.000000	89.900000	3801.700000
max	1.000000	72.000000	118.750000	8684.800000






```
==================================================
FEATURE PREPARATION
==================================================
Features shape: (7021, 30)
```

```
Target shape: (7021,)
Training records: 5616
Testing records: 1405

Logistic Regression trained successfully!
Random Forest trained successfully!

Classification Report: Logistic Regression
              precision    recall  f1-score   support

    No Churn       0.84      0.90      0.87      1033
       Churn       0.66      0.53      0.59       372

    accuracy                           0.80      1405
   macro avg       0.75      0.72      0.73      1405
weighted avg       0.79      0.80      0.80      1405

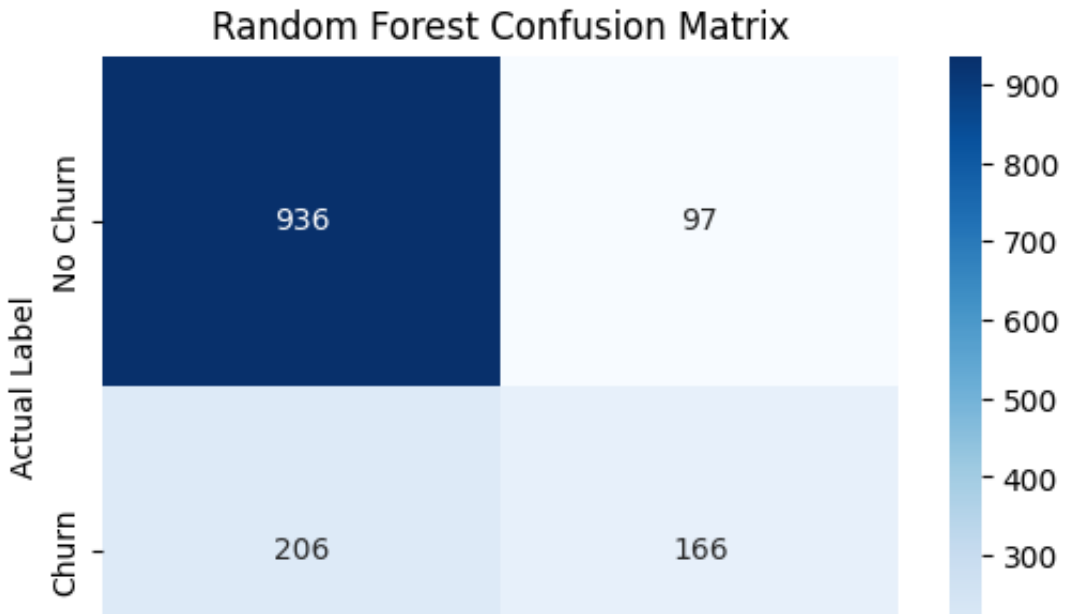

Classification Report: Random Forest
              precision    recall  f1-score   support

    No Churn       0.82      0.91      0.86      1033
       Churn       0.63      0.45      0.52       372

    accuracy                           0.78      1405
   macro avg       0.73      0.68      0.69      1405
weighted avg       0.77      0.78      0.77      1405


MODEL COMPARISON
```

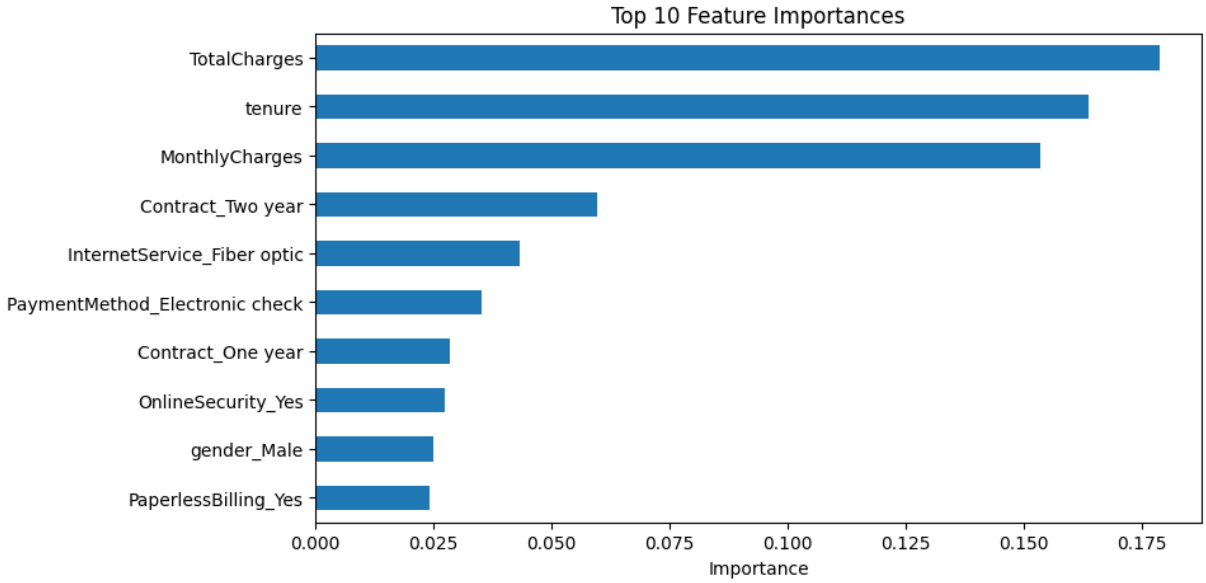
	Model	Accuracy	Precision	Recall	F1 Score	ROC-AUC
0	Logistic Regression	0.804	0.662	0.527	0.587	0.840
1	Random Forest	0.784	0.631	0.446	0.523	0.819





TOP 10 IMPORTANT FEATURES

	Importance
TotalCharges	0.178531
tenure	0.163653
MonthlyCharges	0.153400
Contract_Two year	0.059555
InternetService_Fiber optic	0.043284
PaymentMethod_Electronic check	0.035018
Contract_One year	0.028341
OnlineSecurity_Yes	0.027262
gender_Male	0.024814
PaperlessBilling_Yes	0.024210




```
==================================================
CUSTOMER CHURN CLASSIFIER - PROJECT SUMMARY
==================================================
Total customers: 7021
Training records: 5616
Testing records: 1405
Best model by F1-score: Logistic Regression
Accuracy: 0.804
Precision: 0.662
```

```
Recall: 0.527
F1-score: 0.587
ROC-AUC: 0.84

Model comparison saved as churn_model_comparison.csv
Complete machine learning pipeline executed successfully!
```